

PRACTICAL WEEK-1

Session-1:

Task-1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

- a) Write a C code to create a process using fork() system call.
- b) Read the user input as shell command.
- c) Use a system call exec() to execute the shell command.
- d) Use a wait() system call for waiting a parent process.
- e) Display the PIDs of parent and child processes

OUTPUT and OBSERVATION:

```
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week1$ cd /mnt/d/OSSP_2520080069/Practical/Week1
ananya@Ananya:/mnt/d/OSSP_2520080069/Practical/Week1$ nano task1.c
ananya@Ananya:/mnt/d/OSSP_2520080069/Practical/Week1$ gcc task1.c -o task1
ananya@Ananya:/mnt/d/OSSP_2520080069/Practical/Week1$ ./task1
bash: ./task1: No such file or directory
ananya@Ananya:/mnt/d/OSSP_2520080069/Practical/Week1$ ./task1
Enter a Linux command: ls
Parent PID = 703
Child PID = 704
Parent waiting for child...
Child PID = 704
Executing command...
task1 task1.c
Child process completed.
ananya@Ananya:/mnt/d/OSSP_2520080069/Practical/Week1$ █
```

The program successfully accepted a Linux command from the user and created a child process using fork(). The child process executed the entered command using exec(), while the parent process waited for the child using wait(). The parent and child PIDs were displayed, demonstrating process creation, command execution, and parent-child synchronization in Linux.

Task-2: Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

OUTPUT and OBSERVATION:

```
ananya@Ananya: /mnt/d/OSSP_2520080069/Practical/Week1$ uname -a
Linux Ananya 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UT
C 2026 x86_64 GNU/Linux
ananya@Ananya: /mnt/d/OSSP_2520080069/Practical/Week1$ lscpu
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:               39 bits physical, 48 bits virtual
Byte Order:                  Little Endian
CPU(s):                      8
On-line CPU(s) list:         0-7
Vendor ID:                   GenuineIntel
Model name:                  11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz
CPU family:                  6
Model:                      140
Thread(s) per core:          2
Core(s) per socket:          4
Socket(s):                   1
Stepping:                    1
BogoMIPS:                    4838.40
Flags:                       fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov p
at pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdt
scp lm constant_tsc arch_perfmon rep_good nopl xtopology tsc_re
liable nonstop_tsc cpuid tsc_known_freq pni pclmulqdq vmx ssse3
fma cx16 pcdm pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadl
ine_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dno
wprefetch ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow ept vpi
d ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid a
vx512f avx512dq rdseed adx smap avx512ifma clflushopt clwb avx5
12cd sha_ni avx512bw avx512vl xsaveopt xsavec xgetbv1 xsaves vn
mi avx512vbmi umip avx512_vbmi2 gfni vaes vpclmulqdq avx512_vnn
i avx512_bitalg avx512_vpopcntdq rdpid movdiri movdir64b fsrm a
vx512_vp2intersect md_clear ibt flush_l1d arch_capabilities

Virtualization features:
Virtualization:              VT-x
Hypervisor vendor:           Microsoft
Virtualization type:         full
Caches (sum of all):
L1d:                          192 KiB (4 instances)
L1i:                          128 KiB (4 instances)
L2:                           5 MiB (4 instances)
L3:                           8 MiB (1 instance)
NUMA:
NUMA node(s):                 1
NUMA node0 CPU(s):           0-7
Vulnerabilities:
Gather data sampling:         Not affected
Ghostwrite:                   Not affected
Indirect target selection:    Mitigation; Aligned branch/return thunks
Itlb multihit:               Not affected
L1tf:                         Not affected
Mds:                          Not affected
```

```
Ubuntu
Meltdown: Not affected
Mmio stale data: Not affected
Old microcode: Not affected
Reg file data sampling: Not affected
Retbleed: Mitigation; Enhanced IBRS
Spec rstack overflow: Not affected
Spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1: Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2: Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBRSE-eIBRS SW sequence; BHI SW loop, KVM SW loop
Srbds: Not affected
Tsa: Not affected
Tsx async abort: Not affected
Vmscape: Not affected
ananya@Ananya:/mnt/d/OSSP_2520080069/Practical/Week1$ ps
  PID TTY          TIME CMD
  447 pts/0        00:00:00 bash
  606 pts/0        00:00:00 bash
  711 pts/0        00:00:00 ps
ananya@Ananya:/mnt/d/OSSP_2520080069/Practical/Week1$ top
top - 17:21:42 up 37 min,  1 user,  load average: 0.00, 0.00, 0.00
Tasks:  26 total,   1 running,  25 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  2.1 sy,   0.0 ni, 97.9 id,   0.0 wa,   0.0 hi,   0.0 si,   0.0 st
MiB Mem :  3749.6 total,  3145.9 free,   425.0 used,   253.3 buff/cache
MiB Swap: 1024.0 total, 1024.0 free,    0.0 used.  3324.7 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
    1 root        20   0   24236 15432 11628 S   0.0   0.4   0:00.75 systemd
    2 root        20   0   3180  2208  2072 S   0.0   0.1   0:00.01 init-systemd(Ub
    7 root        20   0   3180  2040  1940 S   0.0   0.1   0:00.00 init
   47 root        19  -1  50396 16796 15516 S   0.0   0.4   0:00.24 systemd-journal
   64 systemd+    20   0  22540 14648 12028 S   0.0   0.4   0:00.08 systemd-resolve
   87 root        20   0  35268 12320  9320 S   0.0   0.3   0:00.46 systemd-udevd
  107 root        20   0   2888  2008  1880 S   0.0   0.1   0:00.05 chronyd-starter
  108 root        20   0   4472  3192  2948 S   0.0   0.1   0:00.02 cron
  109 message+    20   0   8816  5460  4780 S   0.0   0.1   0:00.04 dbus-daemon
  142 root        20   0  44092 29820 14680 S   0.0   0.8   0:00.23 networkd-dispat
  146 root        20   0  18432  9476  8236 S   0.0   0.2   0:00.10 systemd-logind
  166 syslog      20   0 220548  5888  4712 S   0.0   0.2   0:00.07 rsyslogd
  194 root        20   0   5492  2796  2612 S   0.0   0.1   0:00.01 agetty
  217 _chrony      20   0  20420  6472  5696 S   0.0   0.2   0:00.09 chronyd
  223 _chrony      20   0  12092  2516  1872 S   0.0   0.1   0:00.00 chronyd
  263 root        20   0 123456 32716 18108 S   0.0   0.9   0:00.19 unattended-upgr
  305 root        20   0   8648  5652  4808 S   0.0   0.1   0:00.00 login
  358 ananya     20   0  22336 13664 11040 S   0.0   0.4   0:00.17 systemd
  360 ananya     20   0  22912  4088  2084 S   0.0   0.1   0:00.00 (sd-pam)
  386 ananya     20   0   6136  5480  3928 S   0.0   0.1   0:00.03 bash
  441 root        20   0   3184  1120   980 S   0.0   0.0   0:00.00 SessionLeader
  443 root        20   0   3200  1256  1108 S   0.0   0.0   0:00.06 Relay(447)
  447 ananya     20   0   6268  5600  3920 S   0.0   0.1   0:00.07 bash
```

Linux terminal commands such as `uname`, `lscpu`, `ps`, and `top` were used to examine the operating system, CPU resources, and running processes. The commands demonstrated how Linux manages hardware resources through OS services. CPU resources are managed through process scheduling, memory is abstracted through virtual memory, storage is represented using files and directories, and I/O devices are accessed through device drivers and system calls. Thus, the operating system provides an abstraction layer that allows applications to use hardware resources without directly managing the underlying hardware

